Tecnologias Orientadas A Objetos

Avance 2 del Proyecto

Pablo De Jesús Salazar Rioú —-- Clave: 350287

Froylan Eloy Hernandez Castro

Fecha de Entrega: 18 de septiembre del 2026

Cambios Agregados al Uml

Cambios echos en las clases:

En la clase Sistema:

- **Antes**
  - `cines : List<Cine>`
  - **catalogoPeliculas : List<Pelicula>**
- **Ahora**
  - `cines : Map<String, Cine>` (aqui la clave es el nombre del cine)
  - `catalogoPeliculas : Map<String, Pelicula>` (aqui la clave es el título de la película)

En la clase `Cine`:

- **Antes:** `salas : List<Sala>`
- **Ahora:** `salas : Map<String, Sala>` asociando ahora `idsala` con su objeto `Sala`).

En la clase `Sala`:

- **Antes:** `asientos : List<Asiento>`
- **Ahora:** `asientos : Map<String, Asiento>` (donde la clave es la coordenada de la butaca `"A1"`).

En la clase `Funcion`:

- **Antes:** `asientos : List<Asiento>`
- **Ahora:** `asientos : Map<String, Asiento>`
- **Método actualizado:** `inicializarAsientos(plantillaAsientos : Map<String, Asiento>) : void` (cada función clona los asientos de la sala a su propio mapa para que no se empalmen en los horarios)

Implementacion de nuevas funciones en el menu

Se añadieron nuevas mini menus para lo que son las salas, peliculas, funciones

`menuAdministrado`: Panel con control de acceso por pin

`menuGestionarCines`: Permite dar de alta cines, listar los que ya existen y eliminar sucursales del mapa

`menuGestionarSalas`: nos permite elegir un cine específico y agregarle salas configurando filas, columnas y precio base.

`menuGestionarPeliculas` Permite registrar películas en el catálogo y darlas de baja.

`menuGestionarFunciones`: Permite añadir funciones al vinculando cine ➜ sala ➜película ➜ fecha y hora.

`menuCliente`: Permite consultar cartelera y hacer el flujo de compra seleccionando película, butaca y forma de pago.

Pruebas Unitarias (Junit 5) y Excepciones}

Metodos criticos

Método 1: `Asiento.vender()`

Por que es critico?

Por que no podemos vender un mimo asiento dos veses por una simple razón, por el simple echo de que en un mismo asiento mo puede haber dos personas, ejemplo la persona A compra el asiento A1 y lo paga y llega una persona B queriendo comprar este mismo asiento si esta lograra comprarlo al llegar a dicho asiento ya habria una persona sentado en el.Por esto este método debe garantizar que al compra un boleto este cambia a vendido y si alguien intenta comprar este otra vez se lance Asiento Ocupado Exception.

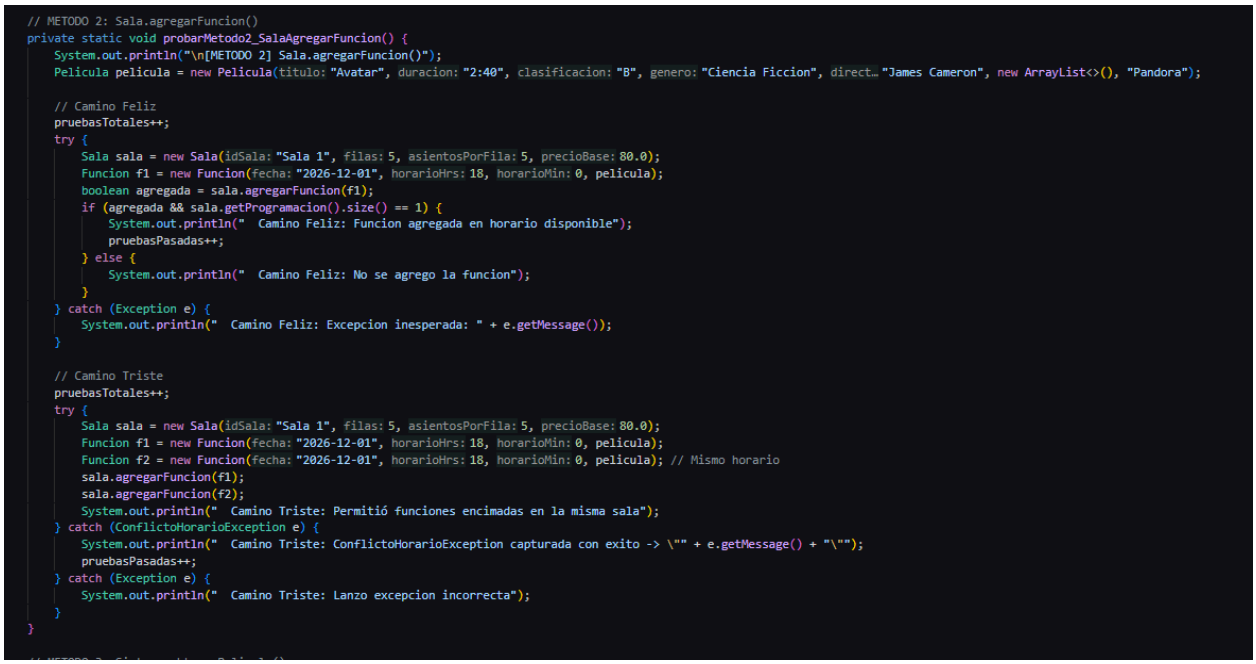
```java
// METODO 1: Asiento.vender()
private static void probarMetodo1_AsientoVender() {
    System.out.println("\n[METODO 1] Asiento.vender()");
    // Camino Feliz
    pruebasTotales++;
    try {
        Asiento a = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        boolean vendido = a.vender();
        if (vendido && a.getEstadoAsiento().equals("VENDIDO")) {
            System.out.println("  Camino Feliz: Asiento disponible vendido exitosamente");
            pruebasPasadas++;
        } else {
            System.out.println(" Camino Feliz: No cambio el estado correctament");
        }
    } catch (Exception e) {
        System.out.println("  Camino Feliz: Excepcion inesperada: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        Asiento a = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        a.vender();
        a.vender(); // Debe lanzar excepcion
        System.out.println("   Camino Triste: Permitió sobreventa del asiento.");
    } catch (AsientoOcupadoException e) {
        System.out.println("  [Camino Triste: AsientoOcupadoException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    } catch (Exception e) {
        System.out.println("   Camino Triste: Lanzo excepcion incorrecta: " + e.getClass().getSimpleName());
    }
}
```

Método 2: `Sala.agregarFuncion(nuevaFuncion)`:

Por que es critico?

Por que en un mimo horario no puede haber 2 peliculas proyectándose al mismo tiempo, asi que si pasa esto se debería lanzar ConflictoHorarioException



```java
// METODO 2: Sala.agregarFuncion()
private static void probarMetodo2_SalaAgregarFuncion() {
    System.out.println("\n[METODO 2] Sala.agregarFuncion()");
    Pelicula pelicula = new Pelicula(titulo: "Avatar", duracion: "2:40", clasificacion: "B", genero: "Ciencia Ficcion", direct… "James Cameron", new ArrayList<>(), "Pandora");

    // Camino Feliz
    pruebasTotales++;
    try {
        Sala sala = new Sala(idSala: "Sala 1", filas: 5, asientosPorFila: 5, precioBase: 80.0);
        Funcion f1 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, pelicula);
        boolean agregada = sala.agregarFuncion(f1);
        if (agregada && sala.getProgramacion().size() == 1) {
            System.out.println("  Camino Feliz: Funcion agregada en horario disponible");
            pruebasPasadas++;
        } else {
            System.out.println("  Camino Feliz: No se agrego la funcion");
        }
    } catch (Exception e) {
        System.out.println("  Camino Feliz: Excepcion inesperada: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        Sala sala = new Sala(idSala: "Sala 1", filas: 5, asientosPorFila: 5, precioBase: 80.0);
        Funcion f1 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, pelicula);
        Funcion f2 = new Funcion(fecha: "2026-12-01", horarioHrs: 18, horarioMin: 0, pelicula); // Mismo horario
        sala.agregarFuncion(f1);
        sala.agregarFuncion(f2);
        System.out.println("  Camino Triste: Permitió funciones encimadas en la misma sala");
    } catch (ConflictoHorarioException e) {
        System.out.println("  Camino Triste: ConflictoHorarioException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    } catch (Exception e) {
        System.out.println("  Camino Triste: Lanzo excepcion incorrecta");
    }
}
```

Método 3:`Sistema.obtener Pelicula(titulo)`:

Por que es critico?

Por que casi todas las funciones que maneja el usuario o cliente se trata de la búsqueda y en este caso es la búsqueda de películas, si se diera el caso de que tecleara mal una tecla y no escribiera correctamente el nombre de la película la pantalla no debería de quedarse en blanco s i no que debería de lanzar un mensaje de EntidadNoEncontradaException

```java
// METODO 3: Sistema.obtenerPelicula()
private static void probarMetodo3_SistemaObtenerPelicula() {
    System.out.println("\n[METODO 3] Sistema.obtenerPelicula()");
    Sistema sis = new Sistema();
    Pelicula p = new Pelicula(titulo: "Batman", duracion: "2:10", clasificacion: "B", genero: "Accion", direct… "Matt Reeves", new ArrayList<>(), "Gotham");
    sis.agregarPelicula(p);

    // Camino Feliz
    pruebasTotales++;
    try {
        Pelicula res = sis.obtenerPelicula(titulo: "Batman");
        if (res != null && res.getTitulo().equals("Batman")) {
            System.out.println("  [OK] Camino Feliz: Pelicula encontrada en el HashMap con exito");
            pruebasPasadas++;
        }
    } catch (Exception e) {
        System.out.println("  Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        sis.obtenerPelicula(titulo: "PeliculaInexistente");
        System.out.println(" Camino Triste: No lanzo excepcion para pelicula no existente");
    } catch (EntidadNoEncontradaException e) {
        System.out.println("   Camino Triste: EntidadNoEncontradaException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    } catch (Exception e) {
        System.out.println("  Camino Triste: Excepcion incorrecta");
    }
}
```

.

Método 4 :`Boleto.calcularTotal()`:

Por que es critico?

Por que si al calcular el total de los boletos comprados se haya calculado por centavos de mas o de menos esto representaría un gran problema tanto para el cine o para el usuario y para que esto no pase se debe sumar con precisión y verificar que las butacas esten disponibles y no tengan precios negativos.

```java
// METODO 4: Boleto.calcularTotal()
private static void probarMetodo4_BoletoCalcularTotal() {
    System.out.println("\n[METODO 4] Boleto.calcularTotal()");
    Pelicula p = new Pelicula(titulo: "Toy Story", duracion: "1:30", clasificacion: "A", genero: "Animacion", direct… "John Lasseter", new ArrayList<>(), "Juguetes");
    Funcion f = new Funcion(fecha: "2026-11-20", horarioHrs: 15, horarioMin: 0, p);

    // Camino Feliz
    pruebasTotales++;
    try {
        Asiento a1 = new Asiento(identificador: "A1", tipoAsiento: "TRADICIONAL", precio: 80.0);
        Asiento a2 = new Asiento(identificador: "A2", tipoAsiento: "TRADICIONAL", precio: 80.0);
        List<Asiento> lista = new ArrayList<>();
        lista.add(a1);
        lista.add(a2);
        Boleto b = new Boleto(p, informacionSala: "Sala 1", f, lista, new Paypal(correo: "pablo@cine.com"));
        double total = b.calcularTotal();
        if (total == 160.0) {
            System.out.println("   Camino Feliz: Total calculado con precision ($" + total + ").");
            pruebasPasadas++;
        } else {
            System.out.println("  Camino Feliz: Calculo erroneo: " + total);
            }
    } catch (Exception e) {
        System.out.println("  Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        Boleto bInvalido = new Boleto(p, informacionSala: "Sala 1", f, new ArrayList<>(), new Paypal(correo: "pablo@cine.com"));
        bInvalido.calcularTotal();
        System.out.println("   Camino Triste: Calculo total con lista de asientos vaci");
    } catch (IllegalStateException e) {
        System.out.println("  Camino Triste: IllegalStateException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    } catch (Exception e) {
        System.out.println(" Camino Triste: Excepcion incorrecta: " + e.getClass().getSimpleName());
    }
}
```

Método 5 :`TarjetaCredito.procesarPago(monto)`:

Por que es critico?

Aqui es donde valida las reglas de los bancos o de las diversas formas de pago osea que debe asegurarse que que el monto sea mayor a 0 o que el numero de la tarjeta cumpla con la estructura requerida que serian de 16 digitos en total, en caso que falle debe lanzar PagoRechazadoException

```java
// METODO 5: TarjetaCredito.procesarPago()
private static void probarMetodo5_TarjetaCreditoProcesarPago() {
    System.out.println("\n[METODO 5] TarjetaCredito.procesarPago()");
    TarjetaCredito tc = new TarjetaCredito(numeroTarjeta: "1234567890123456");

    // Camino Feliz
    pruebasTotales++;
    try {
        boolean pagado = tc.procesarPago(monto: 150.0);
        if (pagado) {
            System.out.println("  Camino Feliz: Pago de monto valido procesado exitosamente");
            pruebasPasadas++;
        }
    } catch (Exception e) {
        System.out.println("   Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        tc.procesarPago(-50.0);
        System.out.println("   Camino Triste: Permitió cobrar un monto negativo");
    } catch (IllegalArgumentException e) {
        System.out.println("  Camino Triste: IllegalArgumentException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    }
}
```

Método 6 :`Sistema.eliminarCine(nombreCine)`:

Por que es critico?

Por que al eliminar un cine se estaría eliminando en cascada las salas de los cines sus funciones sus asientos asi que se debera verificar con exactitud el nombre del cien a eliminar y en caso de que no exista lanzar EntidadNoEncontradaException para no desequilibrar el orden de los demas

```java
// METODO 6: Sistema.eliminarCine()
private static void probarMetodo6_SistemaEliminarCine() {
    System.out.println("\n[METODO 6] Sistema.eliminarCine()");
    Sistema sis = new Sistema();
    sis.agregaCine(new Cine(nombreCine: "Cinepolis Tangamanga"));

    // Camino Feliz
    pruebasTotales++;
    try {
        boolean eliminado = sis.eliminarCine(nombreCine: "Cinepolis Tangamanga");
        if (eliminado && sis.getCines().isEmpty()) {
            System.out.println(" Camino Feliz: Cine eliminado correctamente del sistema");
            pruebasPasadas++;
        }
    } catch (Exception e) {
        System.out.println(" Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        sis.eliminarCine(nombreCine: "CineFantasma");
        System.out.println("   Camino Triste: Elimino un cine que no existia");
    } catch (EntidadNoEncontradaException e) {
        System.out.println("  Camino Triste: EntidadNoEncontradaException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    }
}
```

cines.

Método 7:`Cine.obtenerSala(idSala)`:

Por que es critico?

Por que al programar una función o vender boletos , el sistema busca la sala dentro del mapa de los cines , si no existe la búsqueda debe de fallar de inmediato evitando que se vendan boletos en una sala que ni siquiera existe.

```java
// METODO 7: Cine.obtenerSala()
private static void probarMetodo7_CineObtenerSala() {
    System.out.println("\n[METODO 7] Cine.obtenerSala()");
    Cine cine = new Cine(nombreCine: "Cinemex Plaza");
    Sala sala1 = new Sala(idSala: "Sala VIP 1", filas: 4, asientosPorFila: 4, precioBase: 120.0);
    cine.agregaSala(idSala: "Sala VIP 1", sala1);

    // Camino Feliz
    pruebasTotales++;
    try {
        Sala res = cine.obtenerSala(idSala: "Sala VIP 1");
        if (res != null && res.getIdSala().equals("Sala VIP 1")) {
            System.out.println("   Camino Feliz: Sala encontrada en el mapa del cine");
            pruebasPasadas++;
        }
    } catch (Exception e) {
        System.out.println("   Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        cine.obtenerSala(idSala: "Sala Inexistente");
        System.out.println("   Camino Triste: Retorno sala inexistente");
    } catch (EntidadNoEncontradaException e) {
        System.out.println("   Camino Triste: EntidadNoEncontradaException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    }
}
```

Método 8 :`Administrador.validarPin(pin)`

Por que es critico?

El administrador es el que tiene permisos para toda la estructura del cine salas funciones boletos etc etc se debe verificar que solo el administrador pueda entrar asi evitando que culaquier otra persona que no sea el administrador pueda entrar a modificar la estructura de los cines.

```java
// METODO 8: Administrador.validarPin()
private static void probarMetodo8_AdministradorValidarPin() {
    System.out.println("\n[METODO 8] Administrador.validarPin()");
    Administrador admin = new Administrador(nombre: "Carlos", pinAcceso: "9876");

    // Camino Feliz
    pruebasTotales++;
    try {
        boolean valido = admin.validarPin(pinIngresado: "9876");
        boolean invalido = admin.validarPin(pinIngresado: "1111");
        if (valido && !invalido) {
            System.out.println("   Camino Feliz: PIN validado correctamente (true para coincidencia, false para incorrecto)");
            pruebasPasadas++;
        }
    } catch (Exception e) {
        System.out.println("  Camino Feliz: " + e.getMessage());
    }

    // Camino Triste
    pruebasTotales++;
    try {
        admin.validarPin(pinIngresado: null);
        System.out.println("   Camino Triste: Permitió PIN nulo sin lanzar excepcion");
    } catch (IllegalArgumentException e) {
        System.out.println("  Camino Triste: IllegalArgumentException capturada con exito -> \"" + e.getMessage() + "\"");
        pruebasPasadas++;
    }
}
```